Python Chapter 6 Lists and Tuples Activities

Contents

Python Chapter 6 Lists and Tuples Activities 1
DRY 2
How to Think Like a Computer Scientist (Interactive Edition) 2
Online Tutorials 2
Lists 2
Tuples 3
Traversing a List with a Loop 3
Slice 4
List Operations 5
List Functions 6
- Examples 6
- in 7

List Methods 8
- append() 8
- insert() 8
- sort() 9
- reverse() 9
- index() 9
- remove() 10
- del 10
- pop() 10
- extend() 10

Two-dimensional Lists 11
Tutorial 1: Two-Dimensional Nested List 12
Python Pickle 14
Tutorial 2: Pickle and Unpickle a List 15
Tutorial 3: Pickle a Custom Object 17
Try Except 19

Tutorial 4: Try Except Fahrenheit Converter 20
Assignment 1: Pickle Two-Dimensional List 20
Assignment Submission..... 21

Time required: 90 minutes

DRY

Don't Repeat Yourself

How to Think Like a Computer Scientist (Interactive Edition)

Go through the following tutorials.

Lists

Online Tutorials

Go through the following tutorials.

Lists

- Python Lists
    - Access List Items
    - Change List Items
    - Add List Items
    - Remove List Items
    - Loop Lists
    - List Comprehensions
    - Sort Lists
    - Copy Lists
    - Join Lists
    - List Methods

Tuples

- Python Tuples
    - Access Tuples
    - Update Tuples
    - Unpack Tuples
    - Loop Tuples
    - Join Tuples
    - Tuple Methods

Traversing a List with a Loop

The most common way to iterate or traverse the elements of a list is with a for loop. The syntax is the same as for strings.

```
for cheese in cheeses:
    print(cheese)
cheddar
colby
swiss
```

If we want the index number and the element, we use the enumerate method.

```
for index, cheese in enumerate(cheeses):
    print(index, cheese)
0 cheddar
1 colby
2 swiss
```

If you want to write or update the elements, you need the indices. A common way to do that is to combine the functions **range** and **len**:

```
# range_len.py
numbers = [1, 3, 5]
for i in range(len(numbers)):
    # Update element in list
    numbers[i] = numbers[i] * 2
    print(f"{i + 1} {numbers[i]}")
```

Example run:

```
1 2
2 6
3 10
```

This loop traverses the list and updates each element. **len** returns the number of elements in the list. **range** returns a list of indices from 0 to n − 1, where **n** is the length of the list. Each time through the loop, **i** gets the index of the next element. The assignment statement in the body uses **i** to read the old value of the element and to assign the new value.

Nested List: Although a list can contain another list, the nested list still counts as a single element. The length of this list is four:

```
["spam", 1, ["Brie", "Roquefort", "Pol le Veq"], [1, 2, 3]]
```

Slice

The **slice** operator works on lists. The start is the index of the first element of the slice. The end is the index denoting the end of the slice. The end is not inclusive.

`list_name[start : end]`

```
t = ["a", "b", "c", "d", "e", "f"]
print(t[1:3])
['b', 'c']
print(t[:4])
['a', 'b', 'c', 'd']
print(t[3:])
['d', 'e', 'f']
```

`t[1:3]` slices the index 1 and 2 items.

`t[:4]` slices everything from the beginning of the list to index 3.

`t[3:]` slices everything after and including index 3.

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. If you omit both, the slice is a copy of the whole list.

```
print(t[:])
['a', 'b', 'c', 'd', 'e', 'f']
```

As lists are mutable, it is often useful to make a copy before performing operations that fold, spindle, or mutilate lists.

A slice operator on the left side of an assignment can update multiple elements:

```
t = ["a", "b", "c", "d", "e", "f"]
t[1:3] = ['x', 'y']
print(t)
['a', 'x', 'y', 'd', 'e', 'f']
```

To remove more than one element, you can use **del** with a **slice** index:

```
t = ["a", "b", "c", "d", "e", "f"]
del t[1:5]
print(t)
['a', 'f']
```

slice selects all the elements up to, but not including the second index.

List Operations

Many list operations work the same as string operations.

Expression	Result
[7,8]+[3,4,5]	[7,8,3,4,5]
[7,8]*3	[7,8,7,8,7,8]
[0]*5	[0,0,0,0,0]

The + operator concatenates lists:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b
print(c)
[1, 2, 3, 4, 5, 6]
```

The ***** operator repeats a list a given number of times:

```
alist = [0]
alist = alist * 4
print(alist)
[0, 0, 0, 0]
blist = [1, 2, 3]
blist = blist * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats four times. The second example repeats the list three times.

List Functions

There are many built-in functions that operate on lists without writing your own loops.

Function	Description
`len()`	Returns the number of items in the list
`sum()`	Returns the sum of the items in the list
`min()`	Returns the minimum of the items in the list
`max()`	Returns the maximum of the items in the list
`sorted()`	Returns a sorted list

Examples

```
>>> nums = [3, 41, 12, 9, 74, 15]
>>> print(len(nums))
6
>>> print(max(nums))
74
>>> print(min(nums))
3
>>> print(sum(nums))
154
>>> print(sum(nums)/len(nums))
25
>>> print(sorted(nums))
[3, 9, 12, 15, 41, 74]
```

The **sum()** function only works when the list elements are numbers. The other functions (**max()**, **len()**, etc.) work with lists of strings and other types that can be comparable.

This is a program that computes the average of a list of numbers entered by the user using a list.

A program to compute an average without a list:

```
total = 0
count = 0
while (True):
    inp = input("Enter a number: ")
    if inp == "done":
        break
    value = float(inp)
    total = total + value
    count = count + 1

average = total / count
print(f"Average: {average}”)
```

In this program, we have count and total variables to keep the number and running total of the user’s numbers as we repeatedly prompt the user for a number.

We could simply remember each number as the user entered it and use built-in functions to compute the sum and count at the end.

```
# Filename: average_numbers_with_list.py
# Create empty list for accumulation
num_list = list()
while (True):
    num = input("Enter a number (Enter to quit): ")
    if num == "":
        break
    # Cast input value to float
    value = float(num)
    # Append value to list
    num_list.append(value)
average = sum(num_list) / len(num_list)
print(f"Average: {average}")
```

Replit [average_numbers_with_list.py]()

We make an empty list before the loop starts, and then each time we have a number, we append it to the list. At the end of the program, we simply compute the sum of the numbers in the list and divide it by the count of the numbers in the list to come up with the average.

in

The in operator tells you if a list contains something.

```
if 2 in L:
   print("Your list contains the number 2. ")
if 0 not in L:
   print("Your list has no zeroes.")
```

List Methods

Python provides methods that operate on lists. Methods use the **.** (dot) operator.

Method	Description
`append(x)`	adds x to the end of the list
`sort()`	sorts the list
`count(x)`	returns the number of times x occurs in the list
`index(x)`	returns the location of the first occurrence of x
`reverse()`	reverses the list
`remove(x)`	removes first occurrence of x from the list
`pop(p)`	removes the item at index p and returns its value
`insert(p,x)`	inserts x at index p of the list

Important note: There is a big difference between list methods and string methods: String methods do not change the original string, but list methods do change the original list. To sort a list L, use `L.sort()` and not `L = L.sort()`.

wrong	*right*
`s.replace('X','x')`	`s = s.replace('X','x')`
`L = L.sort()`	`L.sort()`

append()

Append adds a new element to the end of a list:

```
t = ["a", "b", "c"]
t.append('d')
print(t)

['a', 'b', 'c', 'd']
```

insert()

Insert places a new element at the designate index and moves the others aside.

```
t = ["a", "b", "c"]
t.insert(1, "d")
print(t)
['a', 'd', 'b', 'c']
```

sort()

Arranges the elements of the list from low to high:

```
t = ["d", "c", "e", "b", "a"]
t.sort()
print(t)
['a', 'b', 'c', 'd', 'e']
```

Most list methods are void; they modify the list and return None. If you accidentally write `t = t.sort()`, you will be disappointed with the result.

reverse()

The reverse() reverses the original order of the list. It doesn't sort backward alphabetically, it reverses the current order of the list,

```
t = ["d", "c", "e", "b", "a"]
t.reverse()
print(t)
t.sort()
print(t)
['d', 'c', 'e', 'b', 'a']
['a', 'b', 'c', 'd', 'e']
```

index()

Returns the index of the first occurrence on the list of the element that is given to index() as argument. A runtime error is generated if the element is not found on the list.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
print(fruitlist.index("banana"))
1
```

count()

Returns an integer that indicates how often the element that is passed to it as an argument occurs in the list.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
print(fruitlist.count("banana"))
2
```

remove()

Removes an item with a certain value.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
fruitlist.remove("banana")
print(fruitlist)
['apple', 'cherry', 'banana', 'durian']
```

del

del is a function, not a method. **del** removes an item at an index.

```
t = ["a", "b", "c"]
del t[1]
print(t)
['a', 'c']
```

pop()

pop removes an item from a list at a designated index and returns a value. You can assign that value to a variable. If you don't provide an index, it deletes and returns the last element.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
popfruit = fruitlist.pop(1)
print(fruitlist)
print(popfruit)
['apple', 'cherry', 'banana', 'durian']
Banana
```

extend()

Takes a list as an argument and appends all the elements. This example leaves t2 unmodified.

```
t1 = ["a", "b", "c"]
t2 = ["d", "e"]
t1.extend(t2)
print(t1)
['a', 'b', 'c', 'd', 'e']
```

join()

.join() is not a list method, it is a string method. It does work very well with lists. .join() iterates through the list joining each string with the separator.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
separator = " | "
output = separator.join(fruitlist)
print(output)
apple | banana | cherry | banana | durian
```

Two-dimensional Lists

There are several common things that can be represented by two-dimensional lists, like a Tictac-toe board or the pixels on a computer screen. In Python, one way to create a two-dimensional list is to create a list whose items are themselves lists. Here is an example:

```
board = [[1,2,3],
         [4,5,6],
         [7,8,9]]
```

Indexing: We use two indices to access individual items. To get the entry in row r, column c, use the following:

```
board[r][c]
```

Printing a two-dimensional list: To print a two-dimensional list, you can use nested for loops.

The following example prints a 3 x 3 list:

```
for row in range(3):
    for column in range(3):
        print(board[row][column], end=" ")
    print()
```

Example run:

```
1 2 3
4 5 6
7 8 9
```

Another option is to use the **pprint** function of the **pprint** module. This function is used to "pretty-print" its argument. Here is an example to print a list L:

```
from pprint import pprint
pprint(board)
```

The **`pprint`** function can be used to nicely print ordinary lists and other objects in Python.

Working with two-dimensional lists: Nested for loops, like the ones used in printing a two-dimensional list, can also be used to process the items in a two-dimensional list. Here is an example that counts how many entries in a 10 x 5 list are even.

```
count = 0
for row in range(10):
    for column in range(5):
        if L[row][column]%2==0:
            count = count + 1
```

Creating large two-dimensional lists: To create a larger list, you can use a list comprehension like below:

```
L = [[0]*50 for i in range(100)]
```

This creates a list of zeroes with 100 rows and 50 columns.

Picking out rows and columns: To get the rth row of L, use the following:

```
L[r]
```

To get the cth column of L, use a list comprehension:

```
[L[i][c] for i in range(len(L))]
```

Tutorial 1: Two-Dimensional Nested List

Nested lists are lists that contain other lists as their elements. They allow you to create multi-dimensional data structures.

This tutorial provides explanations and examples for creating, accessing, modifying, and iterating through a two-dimensional nested list in Python.

```python
"""
    Name: nested_list.py
    Author:
    Created:
    Purpose:
"""

# Create a nested list
# Create a variable 'matrix' and assign it a list containing other list
matrix = [
    [1, 2, 3],     # First nested list with elements 1, 2, and 3.
    [4, 5, 6],     # Second nested list with elements 4, 5, and 6.
    [7, 8, 9]      # Third nested list with elements 7, 8, and 9.
]

# Access elements in the nested list
# Print the element at the first row, first column (1).
print(matrix[0][0])
# Print the element at the second row, third column (6).
print(matrix[1][2])

# Modify elements in the nested list
# Change the element at the third row, second column to 10.
matrix[2][1] = 10
# Print the modified element (10).
print(matrix[2][1])

# Iterate through the nested list
# Start a loop that iterates through the outer list (rows).
for row in matrix:
    # Start a nested loop that iterates through
    # the inner list (elements in each row).
    for item in row:
        # Print each element followed by a space, without a new line.
        print(item, end=' ')
    # Print a new line after each row.
    print()
```

Example run:

```
# Output:
# Accessing Elements:
# 1
# 6

# Modifying Elements:
# 10

# Iterating Through the List:
# 1 2 3
# 4 5 6
# 7 10 9
```

```
1
6
10
1 2 3
4 5 6
7 10 9
```

Python Pickle

The Python Pickle module is used to perform serialization and deserialization of Python objects. Almost everything in Python is an object, Dictionaries, Lists, Tuples, etc. Unlike saving to a text file, with pickle you can read the object to memory exactly as it was.

Serializing/pickling a Python object is converting it into a byte stream that can be stored in a file or in a string. Pickled data can then be read using a process called deserialization. This is particularly useful when you want to store the state of an object or transfer it between different programs or systems.

Pickling

- **Definition:** Pickling is the process of converting a Python object into a byte stream.
- **Usage:** It is used to serialize (or "flatten") an object into a format that can be stored or transmitted.
- **Module:** The 'pickle' module in Python provides functions for pickling objects.
- **Example:** You can use `pickle.dump()` to serialize an object and save it to a file.

Unpickling

- **Definition:** Unpickling is the process of reconstructing a Python object from a byte stream.
- **Usage:** It is used to deserialize (or "reconstitute") a previously pickled object.
- **Module:** The 'pickle' module also provides functions for unpickling objects.
- **Example:** You can use `pickle.load()` to read a pickled object from a file and reconstruct it as a Python object.

Key points to remember:

- Pickling is a way to store complex data structures like lists, dictionaries, or custom objects.
- Pickled files usually have a ".pkl" or ".pickle" extension.
- Not all Python objects can be pickled. Objects that can be pickled must be serializable, meaning they can be represented as bytes.
- Pickling and unpickling are particularly useful for caching, data persistence, and inter-process communication.

Pickling and unpickling are essential techniques for working with serialized Python objects, allowing you to store, transmit, and restore complex data structures efficiently.

Let’s start with pickling a list.

Tutorial 2: Pickle and Unpickle a List

Enter the following code.

```python
"""
    Name: pickle_list_oz.py
    Author:
    Created:
    Purpose: Save and retrieve a Python object
    using the pickle library.
"""
# Import pickle library
import pickle


def main():
    # An OZ list to pickle
    oz_list = ("Lions", "Tigers", "Bears")
    FILE_NAME = "pickle_list_oz.pkl"

    print("Original OZ List")
    print(oz_list)

    # Pickle the list to a file with pickle.dump
    # with open, opens the file
    # w write file
    # b binary file type
    with open(FILE_NAME, "wb") as file_handle:
        # Write list to file with binary protocol
        pickle.dump(oz_list, file_handle)
    # When the program exits the with block,
    # the file is closed: the file handle resource is released

    print("List dumped to file\n")

    # Unpickle the list from a file with pickle.load
    # r read file
    # b binary file type
    with open(FILE_NAME, "rb") as file_handle:
        oz_unpickled_list = pickle.load(file_handle)

    print("File loaded to list")
    print("Unpickled OZ List")
    print(oz_unpickled_list)


# Call the main function
if __name__ == '__main__':
    main()
```

Example run:

```
Original List:
['Lions', 'Tigers', 'Bears']
List dumped to file

File loaded to list
Unpickled List:
['Lions', 'Tigers', 'Bears']
```

Tutorial 3: Pickle a Custom Object

You can pickle and unpickle a custom object with the pickle library.

```python
"""
    File: pickle_object.py
    Description: Example of pickling and unpickling a custom object
"""
import pickle


# Create a custom class that you want to pickle and unpickle
class Student:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def __str__(self):
        """Return a string representation of the object."""
        return f"Student(name='{self.name}', age={self.age})"


# Create an instance of the custom class and pickle it:
student1 = Student("Alice", 20)
print(f"{student1}")

# Pickle the object and save it to a file
print("Pickle the object")
with open("student.pickle", "wb") as file:
    pickle.dump(student1, file)

# Unpickle the object from the file
with open("student.pickle", "rb") as file:
    student1 = pickle.load(file)

print("Unpickle the object")
# Student1 holds the unpickled object
print(student1)
```

Example run:

```
Student(name='Alice', age=20)
Pickle the object
Unpickle the object
Student(name='Alice', age=20)
```

Try Except

Things go wrong, errors happen, let's learn to handle those exceptional events with try except.

An exception is a signal that a condition has occurred that can't be easily handled using the normal flow-of-control of a Python program. Exceptions are often defined as being "errors" but this is not always the case. All errors in Python are dealt with using exceptions, but not all exceptions are errors.

Go through this short tutorial for a better understanding of exceptions.

- [Python Try Except]

Basic syntax of exception handling

```
try:
    # Code that might raise an exception
except ExceptionType as e:
    # Code to handle the exception
```

Example of

Here is an example of a program without exception handling.

```
imput = input('Enter Fahrenheit Temperature: ')
fahrenheit = float(input)
celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
print(celsius)
```

If the user types in a float, the program works just fine. If the user types in anything else, a **ValueError** exception occurs.

Example run:

```
Enter Fahrenheit Temperature: d
Traceback (most recent call last):
  File "Z:\_WNCC\Computer Science\Assignments Python\0
aries\Tutorials\temperature_converter.py", line 2, in
    fahrenheit = float(input)
ValueError: could not convert string to float: 'd'
```

Let's rewrite this program to catch and handle exceptions.

Tutorial 4: Try Except Fahrenheit Converter

Create and test the following program with correct and incorrect input.

```python
"""
    Name: temperature_converter.py
    Author:
    Created:
    Purpose: Handle ValueError and general exceptions
"""

try:
    # If the user doesn't type a number,
    # an exception is raised
    fahrenheit = float(input("Enter Fahrenheit Temperature: "))

    # These lines execute if an exception isn't raised
    celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
    print(celsius)

except ValueError as v:
    # This is a specific exception
    # You don't have to have a specific exception handler
    # Handle ValueError, print out exception
    print(f"Please enter a whole number. \n{v}")

except Exception as e:
    # Handle any other exception, print out exception message
    # You can just use this general exception handling
    print(f"Something went wrong. \n{e}")
```

Exceptions are now handled.

Example run:

```
Enter Fahrenheit Temperature: b
Please enter a whole number.
could not convert string to float: 'b'
```

Assignment 1: Pickle Two-Dimensional List

This is the example of a two-dimensional list we worked with earlier.

```python
board = [[1, 2, 3],
         [4, 5, 6],
         [7, 8, 9]]

for row in range(3):
    for column in range(3):
        print(board[row][column], end=" ")
    print()
```

Add the following to this program.

- Pickle and unpickle the list
- Use try except for the file handling.
- Use a specific exception for file handling. Unpickle a file that doesn't exist.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the programs.
3. Submit in Blackboard.